2.3. Optimal paths

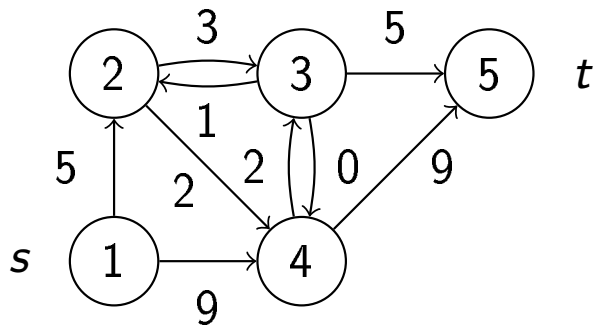
Optimal (shortest, longest, ...) paths have a wide range of applications:

- Google maps, GPS navigators
- planning and management of transportation, electrical and telecommunication networks
- project planning
- VLSI design
- subproblems of more complex problems
- ...

2.3.1 Shortest path problem

Given a directed graph $G = (N, A)$ with a cost $c_{ij} \in \mathbb{R}$ for each arc $(i, j) \in A$, and two nodes s and t, determine a minimum cost (shortest) path from s to t.

- c_{ij} represents the cost (length, travel time,...) of arc $(i, j) \in A$,
- s is the origin, t is the destination.



2.3.2 Dijkstra's algorithm

Definition: A path $\langle (i_1, i_2), (i_2, i_3), \ldots, (i_{k-1}, i_k) \rangle$ is simple if no node is visited more than once.

Property 6

If $c_{ij} \geq 0$ for all $(i, j) \in A$, there is at least one shortest path which is simple.

Assumption: $c_{ij} \geq 0$, for evey $(i, j) \in A$.

Input: $G = (N, A)$ with non-negative arc costs, $s \in N$.

Output: Shortest paths from s to all other nodes of G.

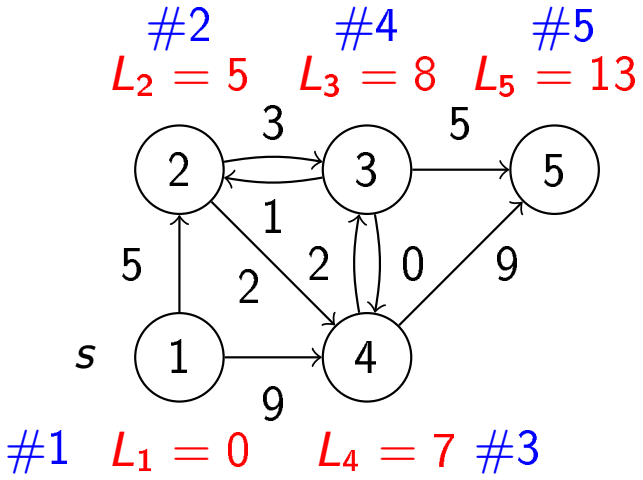
Edsger Dijkstra
(1930-2002)

Idea: Consider the nodes in increasing order of length (cost) of the shortest path from s to any one of the other nodes.

"Greedy" w.r.t. paths from s to j!

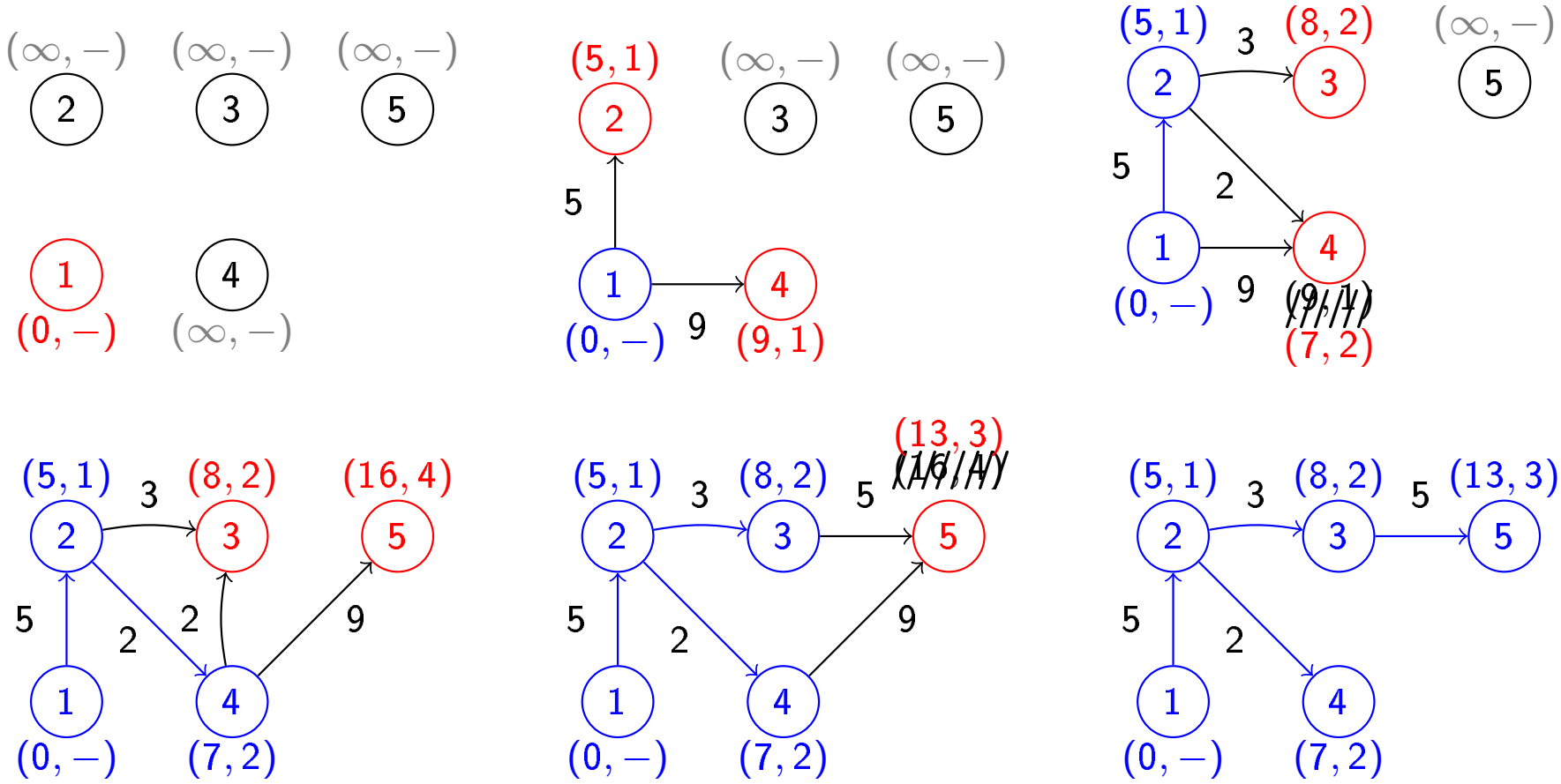
To each node $j \in N$, we assign a label L_j which corresponds, at the end of the algorithm, to the cost of a minimum cost path from s to j.

Example:



- Two labels are associated with each node $j \in N$, $(L_j, pred_j)$, where at the end:
  - L_j = cost of a shortest path from s to j,
  - $pred_j$ = "predecessor" of j in a shortest path from s to j

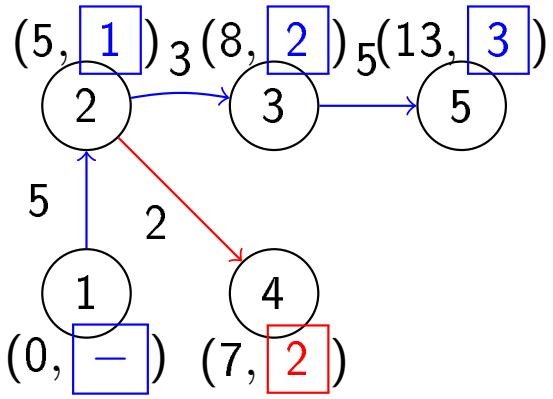
Example:



A set of shortest paths from s to every node $j \neq s$ can be retrieved backwards:

$$pred_j, pred_{pred_j}, \ldots, s.$$

Example



Resulting path from $s = 1$ to $t = 5$: $\langle (1,2),(2,3),(3,5) \rangle$
Resulting path from $s = 1$ to node $j = 4$: $\langle (1,2),(2,4) \rangle$

Data structure

- $S \subseteq N$: subset of nodes whose labels are permanent
- $\overline{S} = (N \setminus S) \subseteq N$: subset of nodes with temporary labels
- $L_j = \begin{cases} \text{cost of a shortest path from } s \text{ to } j, & j \in S \\ \min\{L_i + c_{ij} : (i,j) \in \delta^+(S)\}, & j \notin S \end{cases}$

  (upper bound on the cost of a shortest path)
- $pred_j = \begin{cases} \text{“predecessor” of } j \text{ in the shortest path from } s \text{ to } j, & j \in S \\ u \text{ such that } L_u + c_{uj} = \min\{L_i + c_{ij} : i \in S\}, & j \notin S \end{cases}$

Dijkstra's algorithm

- Input: Graph $G = (N, E)$ with non-negative arc costs, $s \in N$.
- Output: Shortest paths from s to all other nodes of G.

Algorithm 4: Dijkstra's algorithm

```
S ← ∅;
for u ∈ N do
    L_u ← +∞;
    pred_u ← nil;
L_s ← 0;
while |S| ≠ n do
    v ← argmin{L_u : u ∈ S̄};
    S ← S ∪ {v};
    for (v, w) ∈ δ⁺(v) such that L_w > L_v + c_vw do
        L_w ← L_v + c_vw;
        pred_w ← v;
```

If all labels L_u are determined by appropriate updates the overall complexity is $O(n^2)$.

Proposition 2

Dijkstra's algorithm is exact

Proof.

At step k: $S = \{s, i_2, \ldots, i_k\}$ and

$$L_j = \begin{cases} \text{cost of a minimum cost path from } s \text{ to } j, & j \in S \\ \text{cost of a minimum cost path with all intermediate nodes in } S, & j \notin S \end{cases}$$

By induction on k:

- Base case: The statement clearly holds for $k = 1$, since

$$S = \{s\}, \quad L_s = 0, \quad L_j = c_{sj} \;\; \forall j \neq s \;\; (c_{sj} = +\infty \text{ if } (s, j) \notin A).$$

- Inductive step: We verify that, if the statement holds at step k, it must also hold at step $(k + 1)$.

□

Proof.

At step $(k+1)$: Let $v \notin S$ be the node inserted in S and ϕ the path from s to v such that:

$$L_v \leq L_j \quad \forall j \notin S.$$

Let us verify that for every path π from s to v we have $c(\pi) \geq c(\phi)$.

There exist $i \in S$ and $j \notin S$ such that

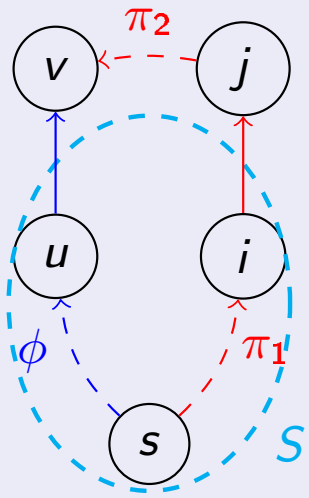
$$\pi = \pi_1 \cup \{(i,j)\} \cup \pi_2$$

where (i,j) is the first arc in $\pi \cap \delta^+(S)$. Moreover,

$$c(\pi) = c(\pi_1) + c_{ij} + c(\pi_2) \geq L_i + c_{ij}$$

since $c(\pi_1) \geq L_i$ and $c(\pi_2) \geq 0$ by inductive assumption.
Thus by the choice of v we have

$$c(\pi) \geq L_i + c_{ij} \geq L_v = c(\phi).$$



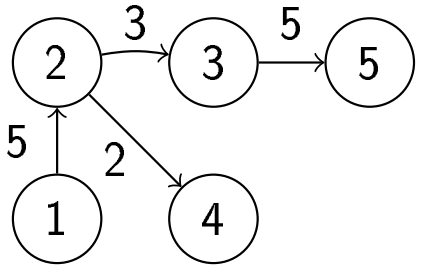
□

Remarks

- A set of shortest paths from s to all the nodes j can be retrieved via the vector of predecessors.

- The union of a set of shortest paths from node s to all the other nodes is an arborescence rooted at s.

  Such arborescences are not minimum cost spanning trees!

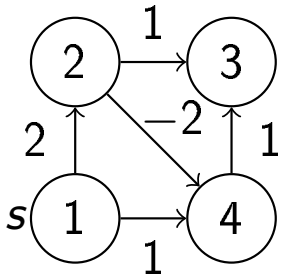


- Dijkstra's algorithm does not work with negative cost arcs

  Example: it yields the path $\langle(1,4),(4,3)\rangle$ of cost 2, instead of $\langle(1,2),(2,4),(4,3)\rangle$.

  Due to $c_{24} < 0$ the last step in the exactness proof fails.

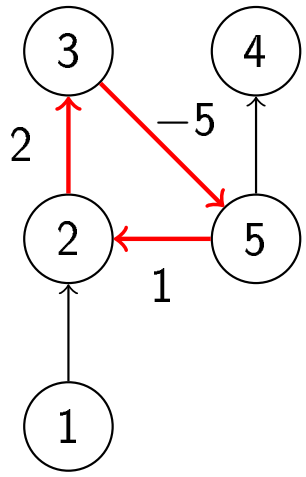
  The cost from 1 to 4 is not updated after the first step.



2.3.3 Shortest path problem with negative costs

Observation: If a graph G contains a circuit of negative cost, the shortest path problem may not be well-defined.

Example



No finite shortest path from s to t.

Circuit's cost: -2

Floyd-Warshall's algorithm:

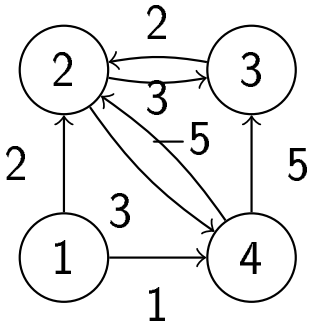
- detects the presence of circuits with negative cost (identifies ill-definedness),
- provides a set of shortest paths between all pairs of nodes, even when there are arcs with negative cost.

Floyd-Warshall's algorithm

- Input: Directed $G = (N, A)$ with an $n \times n$ cost matrix, $C = [c_{ij}]$.
- Output: For each pair of nodes $i, j \in N$, the cost d_{ij} of the shortest path from i to j.
- Data structure: Two $n \times n$ matrices, D, P, whose elements correspond, at the end of the algorithm, to:
  - d_{ij} = cost of a shortest path from i to j;
  - p_{ij} = predecessor of j in that shortest path from i to j.

- **Initialization**

$$d_{ij} = \begin{cases} 0 & i = j \\ c_{ij} & i \neq j \wedge (i,j) \in A, \\ +\infty & \text{otherwise} \end{cases} \qquad p_{ij} = i, i \in N$$

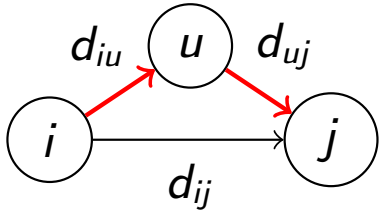


$$D = \begin{bmatrix} 0 & 2 & \infty & 1 \\ \infty & 0 & 3 & 3 \\ \infty & 2 & 0 & \infty \\ \infty & -5 & 5 & 0 \end{bmatrix} \quad P = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & 2 & 2 & 2 \\ 3 & 3 & 3 & 3 \\ 4 & 4 & 4 & 4 \end{bmatrix}$$

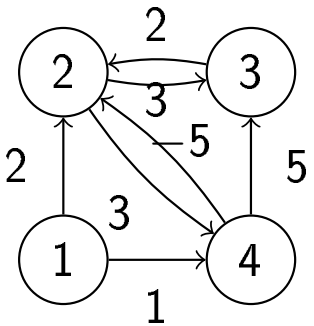
- **Triangular operation**

  For each pair of nodes i, j with $i \neq u$ and $j \neq u$ (including case $i = j$), check whether when going from i to j it is more convenient to go via u:

  $$\text{if } d_{iu} + d_{uj} < d_{ij} \text{ then } d_{ij} \leftarrow d_{iu} + d_{uj}.$$



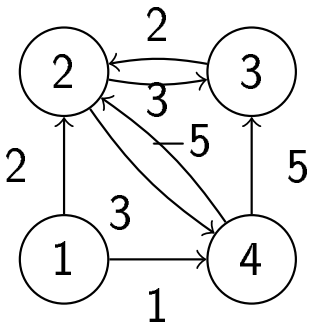
Iteration $u = 1$: Since no such pair exists, the matrices D, P remain unchanged. However, nodes $i = 1$ and $j = 1$ are skipped.



$$D = \begin{bmatrix} 0 & 2 & \infty & 1 \\ \infty & 0 & 3 & 3 \\ \infty & 2 & 0 & \infty \\ \infty & -5 & 5 & 0 \end{bmatrix} \qquad P = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & 2 & 2 & 2 \\ 3 & 3 & 3 & 3 \\ 4 & 4 & 4 & 4 \end{bmatrix}$$

- $0 = d_{22} < d_{21} + d_{12} = \infty + 2$ (no change)
- $3 = d_{23} < d_{21} + d_{13} = \infty + \infty$ (no change)
- $3 = d_{24} < d_{21} + d_{14} = \infty + 1$ (no change)
- $2 = d_{32} < d_{31} + d_{12} = \infty + 2$ (no change)
- $0 = d_{33} < d_{31} + d_{13} = \infty + \infty$ (no change)
- $\infty = d_{34} < d_{31} + d_{14} = \infty + 1$ (no change)
- $-5 = d_{42} < d_{41} + d_{12} = \infty + 2$ (no change)
- $5 = d_{43} < d_{41} + d_{13} = \infty + \infty$ (no change)
- $0 = d_{44} < d_{41} + d_{14} = \infty + 1$ (no change)

Iteration $u = 2$:



$$D = \begin{bmatrix} 0 & 2 & \cancel{\infty}\,\boxed{5} & 1 \\ \infty & 0 & 3 & 3 \\ \infty & 2 & 0 & \cancel{\infty}\,\boxed{5} \\ \infty & -5 & \cancel{5}\,\boxed{-2} & \cancel{0}\,\boxed{-2} \end{bmatrix} \quad P \begin{bmatrix} 1 & 1 & \cancel{1}\,\boxed{2} & 1 \\ 2 & 2 & 2 & 2 \\ 3 & 3 & 3 & \cancel{3}\,\boxed{2} \\ 4 & 4 & \cancel{4}\,\boxed{2} & 4 \end{bmatrix} =$$

- $0 = d_{11} < d_{12} + d_{21} = 2 + \infty$ (no change)
- $\infty = d_{13} > d_{12} + d_{23} = 2 + 3$ ($p_{13} \leftarrow p_{23}$)
- $1 = d_{14} < d_{12} + d_{24} = 2 + 3$ (no change)
- $\infty = d_{31} < d_{32} + d_{21} = 2 + \infty$ (no change)
- $0 = d_{33} < d_{32} + d_{23} = 2 + 3$ (no change)
- $\infty = d_{34} < d_{32} + d_{24} = 2 + 3$ ($p_{34} \leftarrow p_{24}$)
- $\infty = d_{41} < d_{42} + d_{21} = -5 + \infty$ (no change)
- $5 = d_{43} > d_{42} + d_{23} = -5 + 3$ ($p_{43} \leftarrow p_{23}$)
- $0 = d_{44} > d_{42} + d_{24} = -5 + 3$ (Negative cost circuit found! Stop)

Algorithm 5: Floyd-Warshall's algorithm for the all-pairs shortest path problem

for $i \in N$ **do**
  for $j \in N$ **do**
    $p_{ij} \leftarrow i;\ d_{ij} = \begin{cases} 0 & i = j \\ c_{ij} & i \neq j \wedge (i,j) \in A \\ +\infty & \text{otherwise} \end{cases}$

for $u \in N$ `/* Triangular operation */`
do
  for $i \in N \backslash \{u\}$ **do**
    for $j \in N \backslash \{u\}$ **do**
      if $d_{iu} + d_{uj} < d_{ij}$ `/* Update` d_{ij} `if cheaper to go via` u `*/`
      then
        $d_{ij} \leftarrow d_{iu} + d_{uj};\ p_{ij} \leftarrow p_{uj};$

  for $i \in N$ **do**
    if $d_{ii} < 0$ `/* There exists a circuit with negative cost */`
    then Stop

Proposition 3

Floyd-Warshall's algorithm is exact.

Proof's idea:

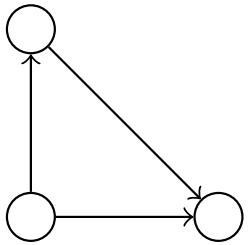
Assume that the nodes of G are numbered from 1 to n.
Verify that, if the node index order is followed, after the u-th cycle the value d_{ij} (for any i and j) corresponds to the cost of a shortest path from i to j with only intermediate nodes in $\{1, \ldots, u\}$.

Complexity

$O(n^3)$ where $n = |N|$

2.3.4 Optimal paths in directed acyclic graphs

Definition: A directed graph $G = (N, A)$ is acyclic (is a DAG) if it contains no circuits.



Problem

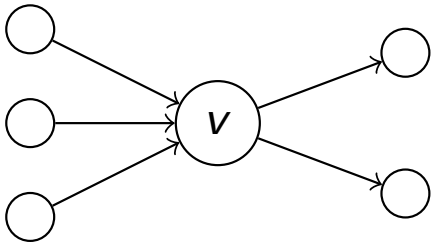
Given a DAG $G = (N, A)$ with a cost $c_{ij} \in \mathbb{R}$ for each $(i, j) \in A$, and nodes s and t, determine a shortest (longest) path from s to t.

Property (Topological order): The nodes of any DAG G can be ordered topologically, that is, indexed so that for each arc $(i, j) \in A$ we have $i < j$.

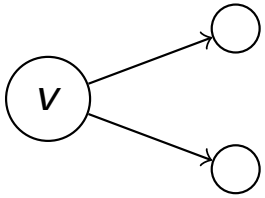
The topological order can be exploited in a very efficient Dynamic Programming algorithm to find shortest (or longest) paths.

Topological ordering method

$G = (N, A)$ is represented via the lists of predecessors $\delta^-(v)$ and successors $\delta^+(v)$ for each node v.



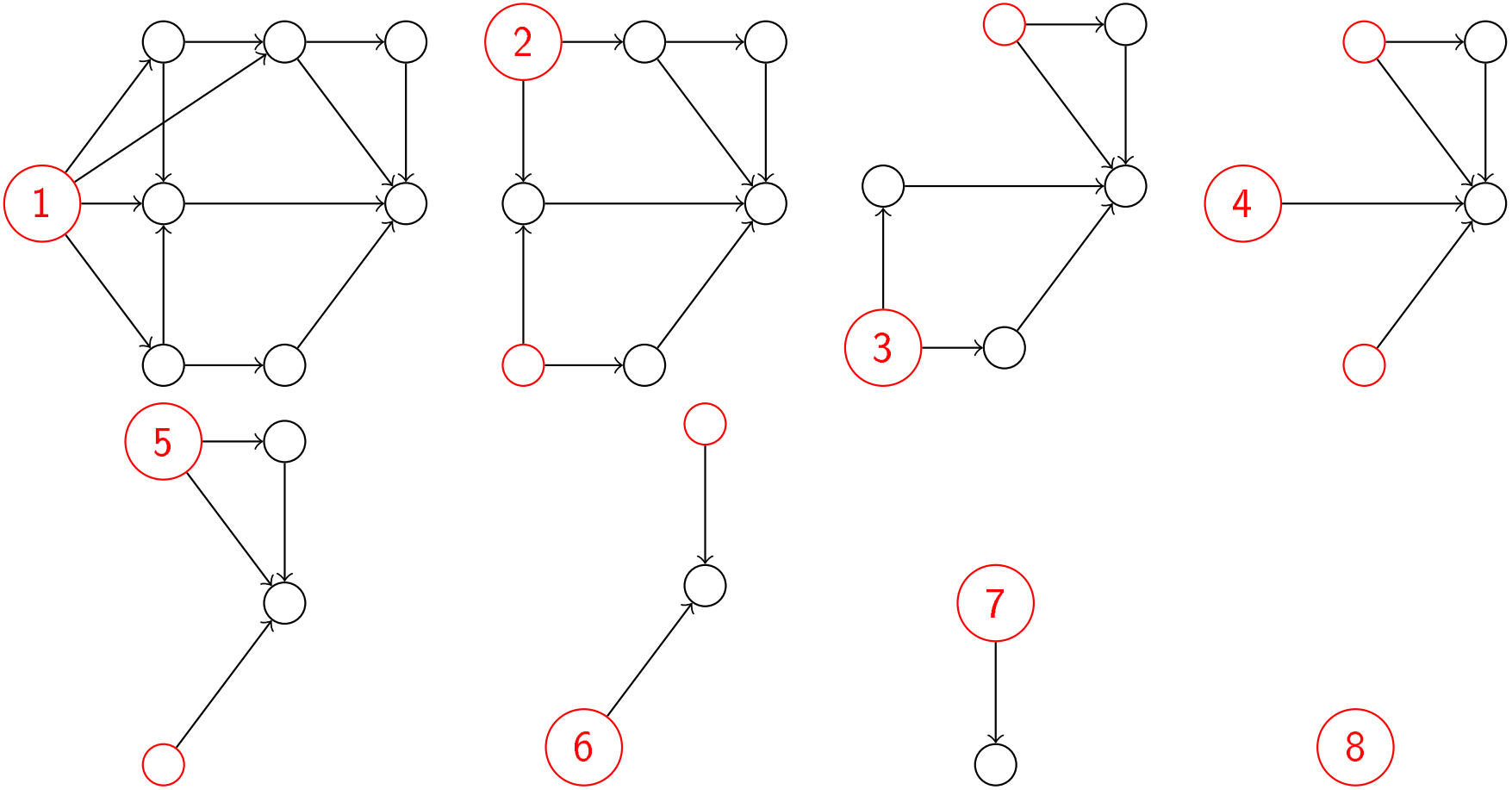
1. Assign the smallest positive integer not yet assigned to a node $v \in N$ with $\delta^-(v) = \emptyset$.



Such a node v exists because G does not contain circuits.

2. Delete the node v with all its incident arcs.
3. Go back to 1. until there are nodes in the current subgraph.

Example



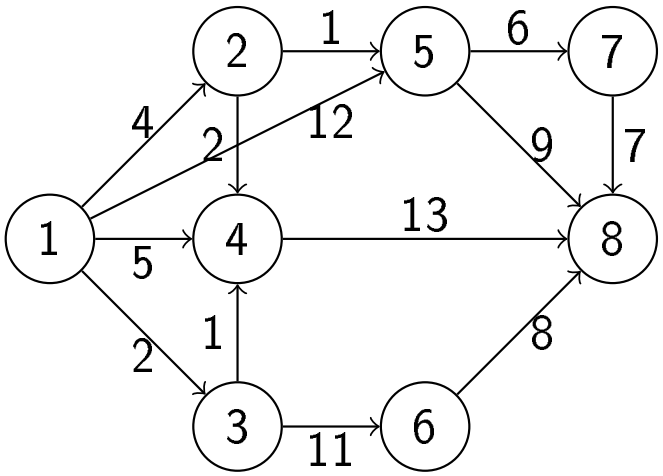
Complexity

$O(n+m)$ where $n = |N|$ and $m = |A|$

because each node/arc is considered at most once.

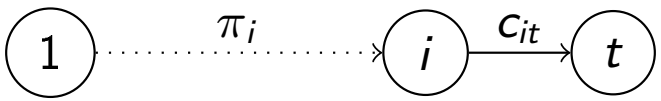
Problem

Given G and a topological order of the nodes, how to find a shortest path from node 1 to node 8?



Dynamic Programming for shortest paths in DAGs

Any shortest path π_t from 1 to t with at least 2 arcs, can be subdivided into two parts: π_i and (i, t), where π_i is a shortest subpath from s to i.



- For each node $i = 1, \ldots, t$, let L_i be the cost of a shortest path from 1 to i. Then

$$L_t = \min_{(i,t) \in \delta^-(t)} \{L_i + c_{it}\},$$

  where the minimum is taken over all possible predecessors i of t.

- If G is a DAG and nodes are topologically ordered, the only possible predeces- sors of t in a shortest path π_t from 1 to t are those with index $i < t$. Thus

$$L_t = \min_{i<t} \{L_i + c_{it}\}.$$

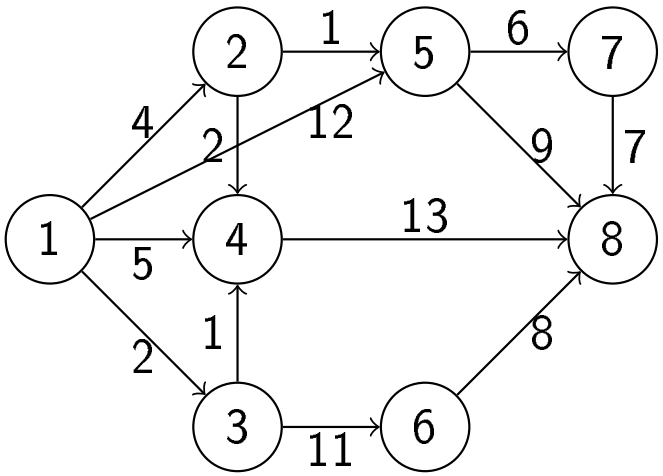
- In a G with circuits, any node different from t can be a predecessor of t in π_t!

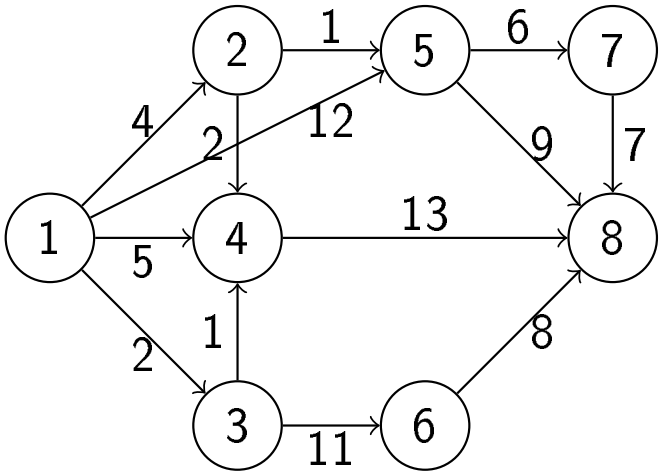
For DAGs whose nodes are topologically ordered, $L_{t-1}, \ldots, L_1$ satisfy the same type of recursive relations:

$$L_{t-1} = \min_{i<t-1} \{L_i + c_{i,t-1}\}; \ \cdots; \ L_2 = \min_{i=1} \{L_i + c_{i2}\} = L_1 + c_{12}; \ L_1 = 0$$

which can be solved in the reverse order:

$$L_1 = 0; \ L_2 = L_1 + c_{12}; \ \cdots; \ L_t = \min_{i<t} \{L_i + c_t\}$$





$L_1 = 0$ $\qquad pred_1 = 1$

$L_2 = L_1 + c_{12} = 4$ $\qquad pred_2 = 1$

$L_3 = L_1 + c_{13} = 2$ $\qquad pred_3 = 1$

$L_4 = \min_{i=1,2,3}\{L_i + c_{i4}\} = \min\{0+5, 4+2, 2+1\} = 3$ $\qquad pred_4 = 3$

$L_5 = \min_{i=1,2}\{L_i + c_{i5}\} = \min\{0+12, 4+1\} = 5$ $\qquad pred_5 = 2$

$L_6 = L_3 + c_{36} = 2 + 11 = 13$ $\qquad pred_6 = 3$

$L_7 = L_5 + c_{57} = 5 + 6 = 11$ $\qquad pred_7 = 5$

$L_8 = \min_{i=4,5,6,7}\{L_i + c_{i8}\} = \min\{3+13, 5+9, 13+8, 11+7\} = 14$ $\qquad pred_8 = 5$

$L_1 = 0$ $\quad pred_1 = 1$

$L_2 = L_1 + c_{12} = 4$ $\quad pred_2 = 1$

$L_3 = L_1 + c_{13} = 2$ $\quad pred_3 = 1$

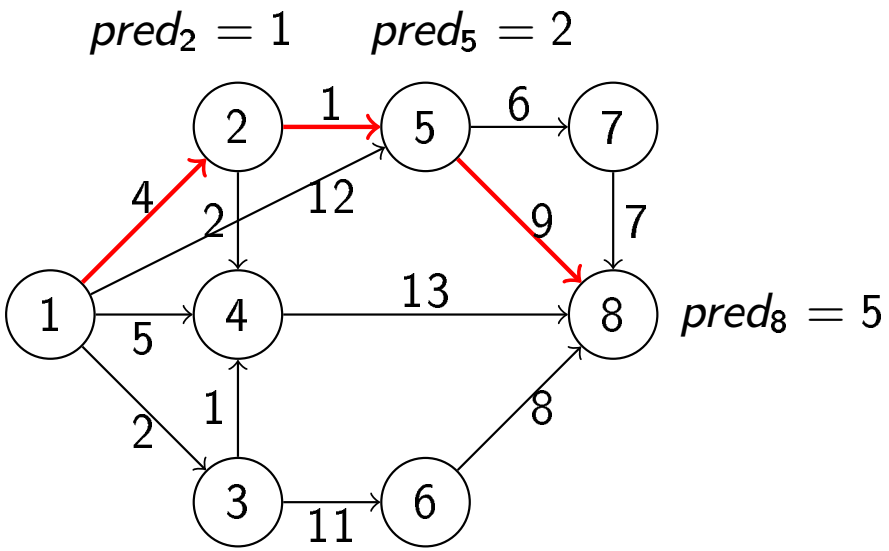
$L_4 = \min_{i=1,2,3}\{L_i + c_{i4}\} = \min\{0+5, 4+2, 2+1\} = 3$ $\quad pred_4 = 3$

$L_5 = \min_{i=1,2}\{L_i + c_{i5}\} = \min\{0+12, 4+1\} = 5$ $\quad pred_5 = 2$

$L_6 = L_3 + c_{36} = 2 + 11 = 13$ $\quad pred_6 = 3$

$L_7 = L_5 + c_{57} = 5 + 6 = 11$ $\quad pred_7 = 5$

$L_8 = \min_{i=4,5,6,7}\{L_i + c_{i8}\} = \min\{3+13, 5+9, 13+8, 11+7\} = 14$ $\quad pred_8 = 5$



Algorithm 6: Dynamic Programming to find shortest paths in DAGs

Sort the nodes of G topologically
$L_1 \leftarrow 0$
for $j = 2, \ldots, n$ `/* in topological order */`
do
  $L_j \leftarrow \min\{L_i + c_{ij} : (i,j) \in \delta^-(j) \wedge (i < j)\}$
  $pred_j \leftarrow v$ such that $(v,j) = \text{argmin}\{L_i + c_{ij} : (i,j) \in \delta^-(j) \wedge (i < j)\}$

Topological ordering of the nodes: $O(n+m)$ with $m = |A|$

Each node/arc is considered exactly once: $O(n+m)$

Overall: $O(n+m)$

Adaptation of DP for longest paths in DAGs

Straightforward adaptation of the DP method to find longest paths in DAGs:

$$L_t = \max_{i<t}\{L_i + c_{it}\}, \ldots$$

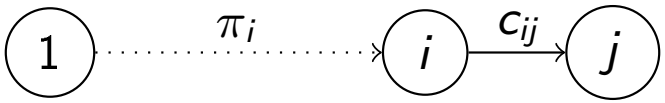
Proposition 4

The Dynamic Programming algorithm for finding shortest (longest) paths in DAGs is exact.

This is due to:

Optimality principle

For any shortest (longest) path π_j from 1 to j there exists $i < j$ such that π_j can be subdivided into two parts: π_i and (i, j), where subpath π_i is of minimum (maximum) length from 1 to i.



Dynamic Programming (DP)

- Proposed in 1953 by Richard Bellman (1920-1984).
- General technique in which an optimal solution, composed of a sequence of elementary decisions, is determined by solving a set of recursive equations.
- DP is applicable to any "sequential decision problem" for which the optimality property is satisfied.
- Wide range of applications: optimal control (rocket launch), equipment maintenance and replacement, selection of inspection points along a production line, ...

Exercise

One machine, 5 year time horizon. Can buy/sell machine at beginning of each year. Cost of a new machine is constant (12 K€).

Goal

Minimize the net total cost (price + maintenance - recover) over 5 year horizon.

Age	Main.	Reco.
0	2	–
1	4	7
2	5	6
3	9	2
4	12	1

If we buy a machine at beginning of 1st year and sell it at the beginning of 2nd year, the net cost is: $12 + 2 - 7 = 7$

If we buy it at the beginning of 1st year and sell it at the beginning of 3rd year, the net cost is: $12 + 2 + 4 - 6 = 12$, etc, ...

Show how the problem of determining an optimal maintenance – replacement policy of minimum total cost (for the next 5 years) can be solved via DP.

Hint

Reduce it to a minimum cost path problem in an appropriate acyclic graph.
Find all the optimal maintenance-replacement policies.